TP LOGIN · PYTHON PURO

phyton usuario.py

Manual del código, explicado simple

Explicación paso a paso del programa en Python: cómo levanta la página, dónde se guardan los usuarios, qué hace cada función y qué tocar para cambiar cuentas, administradores, contraseñas o preguntas.

Archivo: phyton usuario.py

Lenguaje: Python 3 (sin JavaScript)

Se ejecuta en: consola + navegador local

Repo: github.com/ziongarcia02jf/tp-login

Fecha: Noviembre 2026

1. ¿Qué es este programa?

Es **un solo archivo de Python**. Al ejecutarlo, el programa:

1. Levanta un servidor local en `http://127.0.0.1:8000`.
2. Abre el navegador automáticamente.
3. Muestra el login y el panel como una página web.

La página se ve en el navegador, pero **toda la lógica está en Python**: validar la contraseña, comparar las respuestas, agregar usuarios y eliminarlos. **No hay JavaScript.**

> **¿Por qué hay HTML entonces?**
> El navegador solo sabe dibujar HTML. Python arma ese HTML y se lo envía. Es como un papel que Python escribe y el navegador muestra. La programación del sistema (las decisiones, los datos, las validaciones) es 100% Python.

2. Cómo se ejecuta

Opción 1: doble clic

Doble clic en `phyton usuario.py`. Se abre una ventana de consola y, a los pocos segundos, el navegador con la página.

Opción 2: desde la consola

```
python "phyton usuario.py"
```

Para cerrar el servidor, apretar `Ctrl+C` en la ventana de la consola o cerrar esa ventana.

Cuentas de ejemplo

Rol	Usuario	Contraseña	Respuestas de seguridad
Administrador	`admin`	`admin123`	`firulais` / `buenos aires` / `san martin`
Usuario común	`usuario`	`usuario123`	`luna` / `cordoba` / `belgrano`

Dónde se guardan los usuarios

En un archivo `usuarios.json` que se crea al lado del `.py`. Si se borra ese archivo, la próxima vez se crean otra vez las dos cuentas de ejemplo.

3. Mapa del archivo

Bloque	Qué contiene
Imports	`http.server`, `json`, `os`, `webbrowser` y `parse_qs`.
Datos	Ruta del JSON, puerto, estado de la sesión y la lista `usuarios`.
Funciones de apoyo	`normalizar`, `nombre_rol`, `buscar` y `contar_admins`.
Páginas HTML	`pagina`, `lista_usuarios`, `pagina_login`, `pagina_preguntas` y `pagina_panel`.
Servidor	`Manejador`: recibe los pedidos del navegador y responde.
main	Crea el servidor, abre el navegador y queda esperando.

4. Los datos

Los usuarios son una lista de diccionarios. Cada diccionario tiene estos campos:

```
{
    "nombre": "Administrador",
    "usuario": "admin",
    "clave": "admin123",
    "rol": "admin",
    "respuestas": ["firulais", "buenos aires", "san martin"]
}
```

Campo	Qué guarda
`nombre`	El nombre que aparece en el saludo.
`usuario`	El nombre para iniciar sesión.
`clave`	La contraseña.
`rol`	`"admin"` o `"comun"`.
`respuestas`	Las 3 respuestas, en orden.

Cargar y guardar

```
def cargar_usuarios():
    if os.path.exists(ARCHIVO):
        with open(ARCHIVO, "r", encoding="utf-8") as archivo:
            return json.load(archivo)

    lista = [ ... las dos cuentas de ejemplo ... ]
    guardar_usuarios(lista)
    return lista


def guardar_usuarios(lista):
    with open(ARCHIVO, "w", encoding="utf-8") as archivo:
        json.dump(lista, archivo, ensure_ascii=False, indent=2)
```

`json.dump` convierte la lista en texto y la guarda. `json.load` hace lo contrario: lee el texto y devuelve la lista.

El estado de la sesión

```
estado = {"pendiente": None, "actual": None}
```

- `pendiente`: el usuario que ya puso bien la contraseña y está respondiendo las preguntas.
- `actual`: el usuario que ya terminó el login y está dentro del panel.

> **Un solo usuario a la vez**
> Como el servidor es local y para una sola persona, el estado se guarda en una variable global. No hace falta manejar cookies ni sesiones.

5. Funciones de apoyo

`normalizar()`

Pasa el texto a minúsculas, le saca los acentos y los espacios de los bordes. Así `CÓRDOBA`, `cordoba` y `Cordoba` son iguales.

```
def normalizar(texto):
    cambios = {
        "á": "a", "é": "e", "í": "i", "ó": "o", "ú": "u",
        "ü": "u", "ñ": "n",
    }
    texto = texto.lower().strip()
    salida = ""
    for letra in texto:
        salida = salida + cambios.get(letra, letra)
    return salida
```

`cambios.get(letra, letra)` significa: si la letra está en el diccionario, devolvé su reemplazo; si no, devolvé la misma letra.

`nombre_rol()`

Devuelve el nombre lindo del rol para mostrar en la página.

```
def nombre_rol(rol):
    if rol == "admin":
        return "Administrador"
    return "Común"
```

`buscar()` y `contar_admins()`

`buscar` recorre la lista y devuelve el usuario con ese nombre, o `None` si no existe. `contar_admins` cuenta cuántos administradores hay, para no borrar al último.

```
def buscar(nombre):
    for u in usuarios:
        if u["usuario"] == nombre:
            return u
    return None


def contar_admins():
    total = 0
    for u in usuarios:
        if u["rol"] == "admin":
            total = total + 1
    return total
```

6. Las páginas HTML

La función `pagina()` arma el esqueleto común de todas las pantallas: el `<head>`, el estilo y el `<body>`. Después cada pantalla arma su contenido y se lo pasa.

```
def pagina(contenido):
    return f"""<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="utf-8">
<title>Login simple</title>
{ESTILO}
</head>
<body>
{contenido}
</body>
</html>"""
```

`pagina_login()`

Muestra el formulario de usuario y contraseña, el mensaje de error si hubo alguno, y la lista de usuarios del sistema para saber qué escribir.

`pagina_preguntas()`

Muestra las 3 preguntas. Toma el usuario de `estado["pendiente"]` y lo escribe arriba, para que se sepa quién está respondiendo.

`pagina_panel()`

Es la pantalla principal. Muestra el nombre, el usuario y el rol. Si el rol es `"admin"`, agrega la lista de usuarios con un botón **Eliminar** al lado de cada uno y el formulario para agregar. Si es común, no muestra nada de eso.

```
if u["rol"] == "admin":
    contenido = contenido + "<hr><h3>Lista de usuarios</h3>"

    for otro in usuarios:
        contenido = contenido + "<p>" + html.escape(otro["usuario"]) + " ..."

        if otro["usuario"] != u["usuario"]:
            contenido = contenido + (
                '<form method="post" action="/eliminar">'
                '<input type="hidden" name="usuario" value="' + html.escape(otro["usuario"]) + '">'
                '<button>Eliminar</button>'
                '</form>'
            )
```

`html.escape()` convierte caracteres especiales como `<` en texto seguro, para que un nombre raro no rompa la página.

7. El servidor

El navegador le pide páginas al servidor con dos tipos de pedidos: **GET** (mostrar una página) y **POST** (enviar un formulario).

Pedidos GET

Dirección	Qué hace
`/`	Si no hay sesión, muestra el login. Si hay sesión, muestra el panel.
`/salir`	Borra la sesión y vuelve al login.
`/favicon.ico`	Responde vacío, para que el navegador no muestre error.

Pedidos POST

Dirección	Qué hace
`/login`	Revisa usuario y contraseña. Si están bien, pasa a las preguntas.
`/verificar`	Revisa las 3 respuestas. Si están bien, entra al panel.
`/agregar`	Solo admin: agrega un usuario nuevo.
`/eliminar`	Solo admin: elimina un usuario, con protecciones.

Validar el login

```
usuario = campo(datos, "usuario")
clave = campo(datos, "clave")
encontrado = buscar(usuario)

if encontrado is None or encontrado["clave"] != clave:
    self.responder(pagina_login("Usuario o contraseña incorrectos."))
else:
    estado["pendiente"] = encontrado
    self.responder(pagina_preguntas())
```

Validar las respuestas

```
r1 = normalizar(campo(datos, "r1"))
r2 = normalizar(campo(datos, "r2"))
r3 = normalizar(campo(datos, "r3"))

if (r1 != normalizar(u["respuestas"][0]) or
        r2 != normalizar(u["respuestas"][1]) or
        r3 != normalizar(u["respuestas"][2])):
    self.responder(pagina_preguntas("Alguna respuesta es incorrecta."))
else:
    estado["actual"] = u
    estado["pendiente"] = None
    self.responder(pagina_panel())
```

Agregar un usuario

Primero revisa que no falte ningún campo y que el usuario no exista:

```
if (nombre == "" or usuario == "" or clave == "" or
        r1 == "" or r2 == "" or r3 == ""):
    self.responder(pagina_panel("Completá todos los campos."))
elif buscar(usuario) is not None:
    self.responder(pagina_panel("Ese usuario ya existe."))
else:
    usuarios.append({
        "nombre": nombre,
        "usuario": usuario,
        "clave": clave,
        "rol": rol,
        "respuestas": [r1, r2, r3],
    })
    guardar_usuarios(usuarios)
```

Eliminar un usuario

No deja borrar la propia cuenta ni al último administrador:

```
if objetivo is None:
    self.responder(pagina_panel("Ese usuario no existe."))
elif objetivo["usuario"] == u["usuario"]:
    self.responder(pagina_panel("No podés eliminar tu propia cuenta."))
elif objetivo["rol"] == "admin" and contar_admins() == 1:
    self.responder(pagina_panel("No se puede eliminar al último administrador."))
else:
    usuarios.remove(objetivo)
    guardar_usuarios(usuarios)
    self.responder(pagina_panel("Usuario eliminado."))
```

Paso 1
Usuario + contraseña

→

Paso 2
3 preguntas de seguridad

→

Resultado
Panel de admin o de usuario común

8. Qué tocar si quiero cambiar algo

Qué quiero cambiar	Dónde	Qué hacer
Usuarios y claves de ejemplo	`cargar_usuarios()`	Cambiar los valores de la lista. Después borrar `usuarios.json`.
Las 3 preguntas	`pagina_preguntas()`	Cambiar el texto de las preguntas y las `respuestas` de cada usuario.
Hacer administrador a alguien	Diccionario del usuario	Poner `"rol": "admin"`, o crearlo con el combo **Administrador**.
Contraseña mínima	`do_POST`, en `/agregar`	Agregar `elif len(clave) < 4:` con un mensaje.
Textos y títulos	Funciones de páginas	Cambiar los `<h2>`, botones y el `<title>`.
El puerto	`PUERTO`	Cambiar `8000` por otro número, por ejemplo `8080`.
Dónde se guardan los datos	`ARCHIVO`	Cambiar el nombre `usuarios.json`.
Reiniciar la demo	Carpeta del programa	Borrar `usuarios.json` y volver a ejecutar.
Agregar un dato nuevo	Varias partes	Agregarlo al diccionario, al formulario, a `/agregar` y a la lista.

8.1. Cambiar las cuentas de ejemplo

```
lista = [
    {
        "nombre": "Administrador",
        "usuario": "admin",
        "clave": "admin123",
        "rol": "admin",
        "respuestas": ["firulais", "buenos aires", "san martin"],
    },
    {
        "nombre": "Usuario comun",
        "usuario": "usuario",
        "clave": "usuario123",
        "rol": "comun",
        "respuestas": ["luna", "cordoba", "belgrano"],
    },
]
```

> **Muy importante**
> Estos datos solo se crean si no existe `usuarios.json`. Si ya existe, hay que borrarlo para que se vuelvan a crear.

8.2. Cambiar las preguntas

En `pagina_preguntas()` están así:

```
<p>1. ¿Nombre de tu primera mascota?<br><input name="r1"></p>
<p>2. ¿Ciudad donde naciste?<br><input name="r2"></p>
<p>3. ¿Nombre de tu escuela primaria?<br><input name="r3"></p>
```

Si cambiás una pregunta, cambiá también la respuesta en la posición correspondiente de `respuestas`: 0 es la primera, 1 la segunda y 2 la tercera.

9. Cómo abrirlo y editarlo

- Con Visual Studio: **Archivo** → **Abrir** → **Archivo** y elegir el `.py`.
- Con Visual Studio Code: clic derecho → **Abrir con Code**.
- Con el Bloc de notas: clic derecho → **Abrir con** → Bloc de notas.

El archivo también está en el repositorio:

```
https://github.com/ziongarcia02jf/tp-login
```

En esta computadora Python 3 ya está instalado. Si se prueba en otra, hay que instalarlo desde `python.org` y marcar la opción **Add python.exe to PATH**.

10. Limitaciones importantes

- Las contraseñas se guardan **sin cifrar**, a propósito, para que el código sea fácil de estudiar.
- El servidor escucha solo en **esta computadora** (`127.0.0.1`).
- Atiende a **un usuario a la vez**, porque la sesión está en una variable global.
- No tiene recuperación de contraseña.
- No bloquea la cuenta por intentos fallidos.
- Es una **demo escolar**: no sirve para guardar contraseñas reales.

> **Para una versión más segura**
> Habría que guardar las contraseñas con hash y salt, usar HTTPS, manejar sesiones con cookies y una base de datos. Eso ya es otro tema, más avanzado.

11. Resumen rápido

Necesito...	Qué hago
Ejecutar el programa	Doble clic en `phyton usuario.py` o `python "phyton usuario.py"`.
Cerrarlo	`Ctrl+C` en la consola o cerrar esa ventana.
Entrar como admin	`admin` / `admin123` + las 3 respuestas.
Entrar como usuario común	`usuario` / `usuario123` + las 3 respuestas.
Agregar un usuario	Entrar como admin, completar el formulario y pulsar **Agregar**.
Eliminar un usuario	Entrar como admin y pulsar **Eliminar** al lado del usuario.
Reiniciar la demo	Borrar `usuarios.json` y volver a ejecutar.
Cambiar las cuentas de ejemplo	Editar `cargar_usuarios()` y borrar `usuarios.json`.
Cambiar las preguntas	Editar `pagina_preguntas()` y las `respuestas`.